

UNIVERSIDAD TECNOLÓGICA DE PEREIRA

PROGRAMA DE INGENIERÍA ELECTRÓNICA / PROGRAMACIÓN II

Diseño de Retos Interactivos

Módulo: Control de Flujo, Iteraciones (**while**), Funciones Modulares y Expresiones Lambda (*Python Quest*)

Presentado por: Luis Mario Vélez

Fecha de entrega: 28 de septiembre de 2026

Docente: Juan Pablo Tabares

SECCIÓN CLASE 3

Tema: Iteraciones con Bucle **while**, Funciones Definidas (**def**), Parámetros por Defecto, Retorno Múltiple, Argumentos Variables (***args**, ****kwargs**) y Expresiones Anónimas (**lambda**)

Métrica / Enfoque: Aplicación técnica en ingeniería electrónica, automatización de procesos, telemetría IoT, gestión de potencia y robótica industrial.

Reto 1: RELLENAR ESPACIOS (Validación de Lecturas en Variador de Velocidad)

- **Mecánica:** *Fill-in-the-blank* (Completar espacios en blanco).
- **Contexto Industrial:** En un variador de velocidad trifásico, la corriente capturada por los sensores Hall debe ser un valor estricta y verdaderamente positivo para evitar errores de fase.

```
corriente = float(input("Ingrese lectura de corriente de fase (A): "))

# Completa la condición del bucle para reintentar mientras el dato sea inválido
_____ corriente <= 0:
    print("⚠ Error: La corriente debe ser mayor a 0 A.")
    corriente = float(input("Reintente la lectura (A): "))

print(f"✅ Lectura válida registrada: {corriente} A")
```

Opciones de Relleno:

1. Espacio 1: **while** | **if** | **for**

Solución Válida: **while**

Reto 2: MODIFICAR / REFACTORIZAR (Calculadora Modular de Carga de Baterías)

- **Mecánica:** Refactorización de código procedimental a modular.
- **Contexto Industrial:** Convertir un script procedimental de cálculo de capacidad de un banco de baterías UPS en una función reutilizable que retorne el valor calculado mediante return.

```
# CÓDIGO A REFACTORIZAR
nivel_actual = 42.0
capacidad_max = 50.0
porcentaje = (nivel_actual / capacidad_max) * 100
print(f"Carga: {porcentaje}%")
```

Opciones de Modificación:

- **A) (Correcta)**

```
def calcular_carga(actual, maximo):
    return (actual / maximo) * 100

porcentaje = calcular_carga(42.0, 50.0)
```

B)

```
def calcular_carga(actual, maximo):
    print((actual / maximo) * 100)

porcentaje = calcular_carga(42.0, 50.0)
```

C)

```
def calcular_carga(*actual):
    return (actual / 50.0) * 100
```

Retroalimentación Académica:

Correcta: La instrucción **return** es indispensable para transferir el resultado fuera del ámbito local de la función hacia el flujo principal del programa.

Reto 3: SELECCIÓN RÁPIDA (Firma de Funciones con ***args** en Telemetría)

- **Mecánica:** Selección única sobre firmas de funciones.
- **Contexto Empresarial:** Un concentrador IoT de telemetría recibe una cantidad indeterminada de lecturas de voltaje de sensores activos en una subestación eléctrica (pueden ser 3, 8 o 15 sensores).

Enunciado: ¿Cuál es la firma correcta para definir una función capaz de aceptar un número variable de argumentos de manera dinámica?

Opciones:

- A) `def procesar_voltajes(sensores[]):`
- B) `def procesar_voltajes(*voltajes):` (Correcta)
- C) `def procesar_voltajes(lambda voltajes):`
- D) `def procesar_voltajes(voltajes.len()):`

Respuesta Correcta: B (El prefijo ***** empaca los parámetros de entrada en una tupla iterable).

Reto 4: ORDENAR / ARRASTRAR (Consola de Control de Rack de Servidores)

- **Mecánica:** *Drag & Drop* / Reorganizar bloques de código (*Parsons Problem*).
- **Contexto IT/Industrial:** Reorganizar la lógica de control para una consola interactiva que procesa comandos en bucle infinito (**while True**), interrumpe el ciclo con **break** y reintenta con **continue**.

Bloques Desordenados:

1. **break**
2. `print("Cerrando sesión del sistema...")`
3. `while True:`
4. `if comando == "SALIR":`
5. `comando = input("Ingrese comando (MEDIR / SALIR): ").strip().upper()`
6. `print("Procesando temperatura del CPU...")`

Secuencia Correcta Esperada:

```
while True:
    comando = input("Ingrese comando (MEDIR / SALIR): ").strip().upper()
    if comando == "SALIR":
        print("Cerrando sesión del sistema...")
        break
    elif comando == "MEDIR":
```

```
print("Procesando temperatura del CPU...")
else:
    continue
```

Reto 5: DEPURA / CORRIGE (Sintaxis de Expresión Lambda en Escalado ADC)

- **Mecánica:** Identificación y corrección de sintaxis en funciones anónimas.
- **Contexto de Embebidos:** Un programador intentó crear una función **lambda** para convertir la lectura de un canal ADC (0 - 1024) a un rango de Voltaje (0 - 5V), pero cometió un error de sintaxis.

Código con Error:

```
# ERROR DE SINTAXIS EN LAMBDA
adc_a_voltios = lambda(raw_adc) -> return (raw_adc / 1024.0) * 5.0
```

Opciones de Corrección:

- A) `adc_a_voltios = lambda raw_adc: (raw_adc / 1024.0) * 5.0` (Correcta)
- B) `adc_a_voltios = def lambda(raw_adc): return (raw_adc / 1024.0) * 5.0`
- C) `adc_a_voltios = lambda raw_adc return (raw_adc / 1024.0) * 5.0`

Respuesta Correcta: A (Las funciones **lambda** utilizan la sintaxis **lambda parametro: expresion** sin usar la palabra clave **return** ni flechas de retorno).

Reto 6: SELECCIÓN RÁPIDA (Parámetros con Valores por Defecto)

- **Mecánica:** Selección única sobre valores opcionales en funciones.
- **Contexto de Automatización:** Se diseña una función para enviar un pulso de activación a un relé industrial. Por defecto, el tiempo de activación debe ser de **1.0** segundo a menos que se especifique otro valor.

Enunciado: ¿Cómo debe definirse la función para establecer **1.0** como valor por defecto en el parámetro **tiempo**?

Opciones:

- A) `def activar_rele(puerto, tiempo == 1.0):`
- B) `def activar_rele(puerto, tiempo = 1.0):` (Correcta)

- C) `def activar_rele(puerto, default tiempo = 1.0):`
- D) `def activar_rele(puerto, tiempo: default(1.0)):`

Respuesta Correcta: B (Los parámetros por defecto se asignan directamente con el operador = en la cabecera de la función).

Reto 7: DEPURA / CORRIGE (Orden Estricto de Parámetros Posicionales y por Defecto)

- **Mecánica:** Corrección de error de compilación / sintaxis `SyntaxError`.
- **Contexto Industrial:** El controlador de un brazo robótico genera un error al compilar la función de posicionamiento de sus articulaciones.

Código con Error:

```
# ERROR: SyntaxError: non-default argument follows default argument
def mover_articulacion(base=0.0, hombro=90.0, codo):
    print(f"Moviendo a: Base={base}, Hombro={hombro}, Codo={codo}")
```

Opciones de Corrección:

- **A) (Correcta)** Ubicar los parámetros obligatorios (sin valor por defecto) antes de los parámetros opcionales: `def mover_articulacion(codo, base=0.0, hombro=90.0):`
- **B)** Agregar la palabra `optional` antes de los valores.
- **C)** Convertir los parámetros en una tupla fija.

Respuesta Correcta: A (En Python, todos los parámetros obligatorios sin valor por defecto deben declararse siempre antes de los parámetros opcionales con valor por defecto).

Código Predeterminado (Base):

```
# Función incompleta que solo retorna la potencia
def calcular_metricas(voltaje, corriente):
    potencia = voltaje * corriente
    return potencia
```

Solución Esperada (Refactorizada):

```
def calcular_metricas(voltaje, corriente):
    potencia = voltaje * corriente
    resistencia = voltaje / corriente
    return potencia, resistencia # Retorna una tupla implícita

# Desestructuración directa de la tupla retornada
p_activa, r_eq = calcular_metricas(220.0, 10.0)
print(f"Potencia: {p_activa} W | Resistencia: {r_eq} Ω")
```

Reto 9: RELLENAR ESPACIOS (Parámetros Nombrados Variables ****kwargs** en SCADA)

- **Mecánica:** *Fill-in-the-blank* sobre argumentos de palabras clave (****kwargs**).
- **Contexto SCADA:** Una función de registro de alarmas recibe parámetros configurables mediante un diccionario dinámico (****kwargs**).

Código Incompleto:

```
# Completa el operador para empaquetar parámetros nombrados en un diccionario
def registrar_alarma(nivel_critico, _____ parametros_extra):
    print(f"Alarma Nivel: {nivel_critico}")
    for clave, valor in parametros_extra.items():
        print(f" - {clave}: {valor}")

# Invocación con múltiples parámetros nombrados
registrar_alarma("ALTO", zona="Bodega 3", presion_bar=4.2, operador="Tech_01")
```

Opciones de Relleno:

1. Espacio 1: ****** | ***** | **&**

Solución Válida: ****** (El operador ****** empaqueta los argumentos nombrados en un diccionario).

Reto 10: SELECCIÓN RÁPIDA (Ámbito de Variables - Global vs Local)

- **Mecánica:** Múltiple opción sobre alcance (*Scope*).
- **Contexto de Software Embebido:** Se tiene una variable de estado del sistema **ESTADO_SISTEMA = "OK"** a nivel global. ¿Qué palabra reservada debe utilizarse dentro de una función para modificar esa variable global directamente?

Opciones:

- A) **local**
- B) **global** (*Correcta*)
- C) **public**

- D) `extern`

Respuesta Correcta: B (La palabra clave `global` le indica al intérprete de Python que la variable modificada pertenece al ámbito global y no al entorno local de la función).

Reto 11: REFACTORIZAR CON LAMBDA + MAP (Filtrado y Calibración de Sensores)

- **Mecánica:** Transformación funcional compacta de listas de datos.
- **Contexto Industrial:** Se recibe una lista con lecturas descalibradas de temperatura [23.5, 24.1, 22.8]. Se requiere aplicar una corrección offset de +0.5°C a cada elemento usando una función `lambda` integrada en `map()`.

Código Predeterminado:

```
# Enfoque con ciclo for tradicional
lecturas = [23.5, 24.1, 22.8]
calibradas = []
for l in lecturas:
    calibradas.append(l + 0.5)
```

Solución Esperada (Elegante y funcional con `lambda`):

```
lecturas = [23.5, 24.1, 22.8]
calibradas = list(map(lambda temp: temp + 0.5, lecturas))
print(f"Lecturas Calibradas: {calibradas}")
```

Reto 12: EJERCICIO INTEGRADOR AVANZADO (Efecto "Bola de Nieve")

- **Mecánica:** Análisis de flujo, adquisición con `while` y retorno modular estructurado en diccionario.
- **Contexto Integrado (IoT / Planta Industrial):** Un sistema de monitoreo recopila lecturas mediante un ciclo `while`, las procesa usando `*args` en una función modular y genera un reporte formateado.

Código del Reto Integrador:

```
def procesar_planta_iot(nombre_node, *muestras, **metadata):
    """
    Recibe lecturas variables (*args) y metadatos del dispositivo (**kwargs).
    Devuelve un diccionario estructurado de telemetría.
    """
    if not muestras:
        return None
```

```

reporte = {
    "nodo": nombre_node,
    "total_registros": len(muestras),
    "valor_max": max(muestras),
    "valor_min": min(muestras),
    "promedio": round(sum(muestras) / len(muestras), 2),
    "detalles": metadata
}
return reporte

# Bucle de recolección en consola usando while
buffer_muestras = []
print("--- ADQUISICIÓN DE DATOS EN TIEMPO REAL ---")
while len(buffer_muestras) < 4:
    lectura = float(input(f"Sensor #{len(buffer_muestras)+1} (PSI): "))
    if lectura < 0:
        print("⚠ Lectura negativa ignorada por fuera de rango.")
        continue # Salta al siguiente ciclo sin guardar
    buffer_muestras.append(lectura)

# Invocación integrando *args (desempaquetado) y **kwargs
telemetry_final = procesar_planta_iot("NODO_CALDERA_01", *buffer_muestras,
ubicacion="Planta_Alta", responsable="Ing_Velez")

print("\n--- REPORTE TÉCNICO SCADA ---")
for clave, valor in telemetry_final.items():
    print(f"• {clave.upper()}: {valor}")

```